

Phase-Assisted Generation: Final Report

Keshav Krishna Prajesh Kumar Siva Satvik Mandapati
New York University, Building LLM Reasoners — Spring 2026
{kk6081, pg2973, sm12779}@nyu.edu

Abstract

Phase-Assisted Generation (PAG) studies adaptive decoding for diffusion language models. The project began with a trace-analysis pipeline around Dream and LLaDA: collect token-level refinement trajectories, extract stabilization features, run change-point detection (CPD), convert segments into block/phase tuples, and train a next-tuple predictor. The main design lesson from this stage was that CPD is useful for interpreting confidence and stabilization dynamics, but it is an awkward permanent label-construction step when the final scheduler needs block-level controls. The codebase therefore evolved toward an AdaBlock-style, block-native pipeline in which traces directly expose block size and stabilization/refinement behavior. This report template summarizes the project motivation, methodology, implemented experiments, preliminary findings, evaluations and limitations.

1 Introduction

Autoregressive language models decode one token at a time, which makes long reasoning traces expensive. Diffusion language models (dLLMs), including Dream and LLaDA, instead generate or refine multiple tokens through iterative denoising. This creates a natural scheduling question: should every region of an answer receive the same block size and refinement budget, or can the decoder adapt based on local difficulty?

The central hypothesis of PAG is that generation traces contain reusable phase structure. Easy spans should stabilize quickly and can be decoded in larger or cheaper blocks, while hard reasoning spans should receive smaller blocks or more refinement. The goal is not only to detect phases after the fact, but to learn a lightweight predictor that can forecast the next block-level control tuple during generation.

The current repository implements this idea as a staged research system:

```
baseline decoding → trace/signals → phase analysis → phase prediction →  
adaptive scheduling → evaluation.
```

2 Problem Statement

Given a partially generated dLLM trace, PAG aims to predict the next decoding control tuple. In the current implementation, the core tuple is

$$y_t = (\text{block_size}_t, \text{refinement_steps}_t),$$

with newer data paths also supporting richer fields such as NFE, top-1 confidence statistics, digit fraction, delimiter fraction, and stabilization summaries.

—CHECK IF THE FOLLOWING FIELDS DEFINITIONS ARE CORRECT—

Here,

- `nfe`: number of forward evaluations in the block which is essentially the number of refinement steps + 1 (initial forward pass)
- `mean_top1_confidence`: mean of the top-1 token confidence across the block
- `min_top1_confidence`: minimum top-1 token confidence across the block
- `digit_fraction`: fraction of tokens in the block that are digits
- `delimiter_fraction`: fraction of tokens in the block that are delimiters (e.g., parentheses, commas)

At inference time, the predicted tuple should drive adaptive unmasking: block size determines how many tokens are committed together, while the refinement/stabilization component determines how much compute is spent before the block is accepted. The intended evaluation compares adaptive decoding against fixed decoding on quality, latency, number of function evaluations (NFE), and stability.

3 Background

Diffusion language models. Dream and LLaDA show that diffusion-style generation can be applied to language modeling and reasoning. Their iterative refinement traces provide the token-level trajectories used in the first phase of this project.

Semi-autoregressive and block decoding. Block diffusion and related semi-autoregressive methods interpolate between fully autoregressive decoding and fully parallel generation. These methods motivate block-level scheduling as a useful control surface.

Adaptive block inference. AdaBlock-dLLM is the closest prior to the final direction of this project because it adapts block size using semantic or confidence-based signals. Our contribution is to frame the schedule as a learned next-control prediction problem over trace-derived tuples rather than only a reactive heuristic.

AdaBlock Trace Analysis

The traces analyzed in this section are the per-block records produced when AdaBlock-dLLM decodes GSM8K with LLaDA-8B-Instruct. Each line of the resulting JSONL file is a single sample (`sample_id`, `dataset`, `split`) together with an ordered list of tuples, one entry per decoded block. A tuple captures four orthogonal views of that block: its compute footprint (`block_size`, `nfe`), its confidence under the model (`mean_top1_confidence`, `min_top1_confidence`), its refinement dynamics (`mean_stab_step` / `max_stab_step`, `mean_ref_step` / `max_ref_step`, `mean_gap` / `max_gap`), and the lexical content of the decoded tokens (`digit_fraction`, `delimiter_fraction`). Together these fields form the input space that the downstream next-tuple predictor consumes during adaptive scheduling.

Concretely, the dataset is collected by running the AdaBlock-dLLM scheduler (LLaDA-8B variant) over the full GSM8K training split (5,000 problems) and a 200-problem subset of the public test split, plus an additional disjoint 200-problem subset of the GSM8K test split that is held out purely as an evaluation set for the downstream scheduler. At every block boundary the scheduler emits the applied block size and the number of forward evaluations actually consumed; in addition, the diffusion loop is instrumented so that for each masked position we record the step at which its argmax first matches the eventual decoded token (`stab_step`) and the step of its final flip (`ref_step`). Block-level features are then computed by aggregating these per-token quantities together with the softmax top-1 probabilities of the committed tokens and a tokenizer-based classification of decoded surface forms into digits and structural delimiters. The resulting corpus contains 138,901 training blocks and 5,543 test blocks (144,444 total) and is the input to every plot in this section as well as to the trained predictor evaluated later in the report.

Block structure

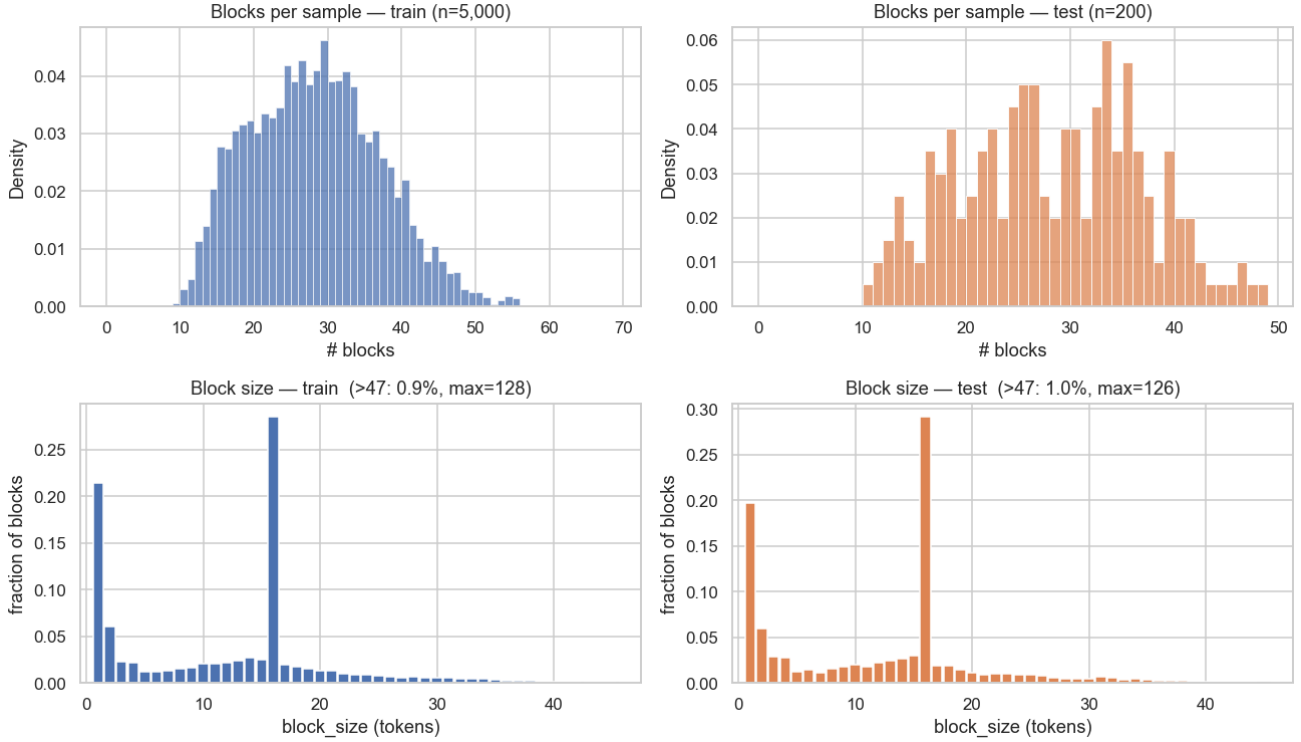


Figure 1: Top: number of blocks AdaBlock-dLLM emits per GSM8K sample, train (left) vs. test (right). Bottom: distribution of `block_size` across all decoded blocks, truncated at the 99th percentile.

Figure 1 shows the decoding granularity AdaBlock-dLLM produces on GSM8K. Blocks per sample (top row) is unimodal around 25–30 with a tail to ~ 55 –60, and the train/test shapes match closely, so the 200-problem evaluation set is representative of the training pool. The `block_size` distribution (bottom row) is sharply bimodal: $\sim 21\%$ of blocks are size 1 (almost always a delimiter the confidence scheduler accepts immediately) and another $\sim 29\%$ sit at the AdaBlock content cap of 16, with sizes 2–15 forming a low decaying ridge. Only $\sim 1\%$ of blocks exceed 47 tokens (max 128), so a discrete classifier capped at 48 classes loses negligible coverage, though it must still handle the heavy point-mass mixture at 1 and 16 rather than a smooth scale.

Compute (NFE) per block

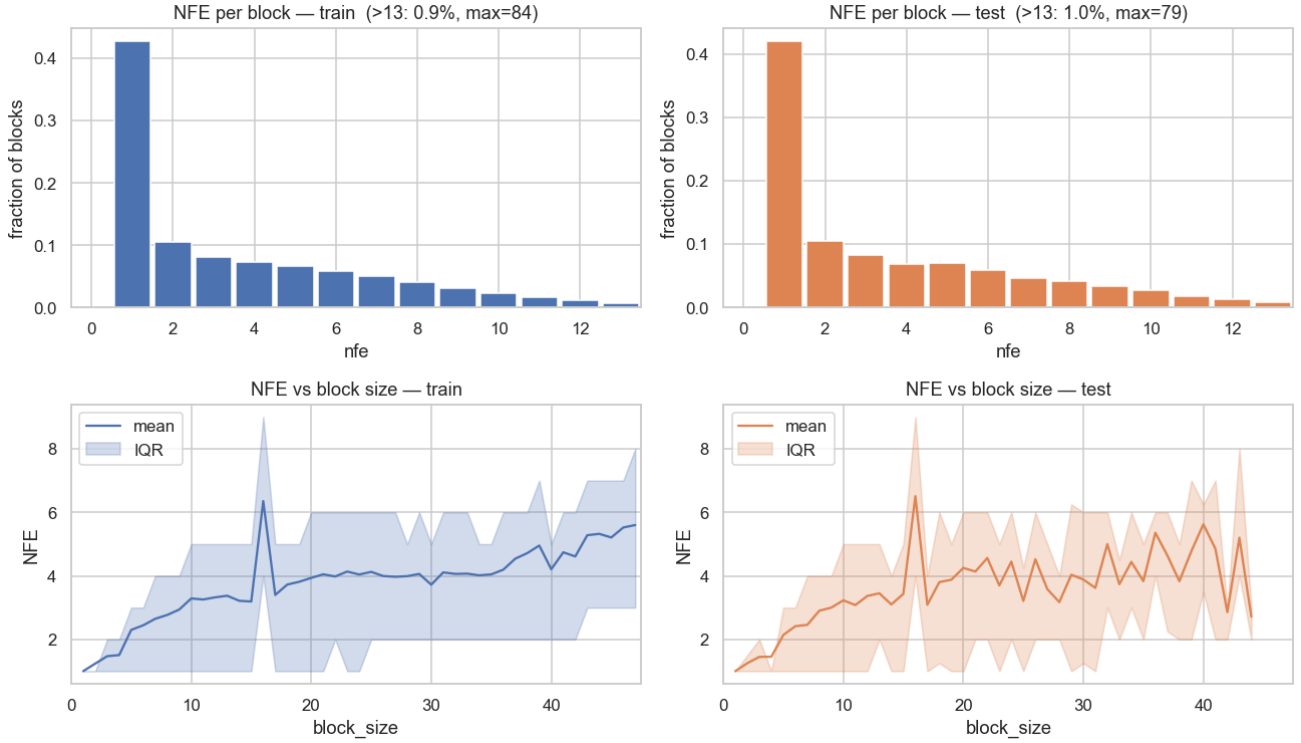


Figure 2: Top: distribution of `nfe` per block, train (left) vs. test (right), truncated at the 99th percentile. Bottom: mean `nfe` as a function of `block_size` with the inter-quartile band, train (left) vs. test (right).

Figure 2 shows the per-block compute spend. About 42% of blocks finish in a single forward pass (the size-1 delimiter blocks), and only $\sim 1\%$ need more than 13 NFE. Conditioning on `block_size` (bottom row), mean NFE grows roughly linearly to ~ 4 at the AdaBlock cap of 16, where a clear spike marks the full-content block, and then plateaus around 4–6. The IQR band is wide at every block size, so `block_size` alone underdetermines compute and a useful predictor must combine it with confidence and refinement signals.

Confidence vs. compute

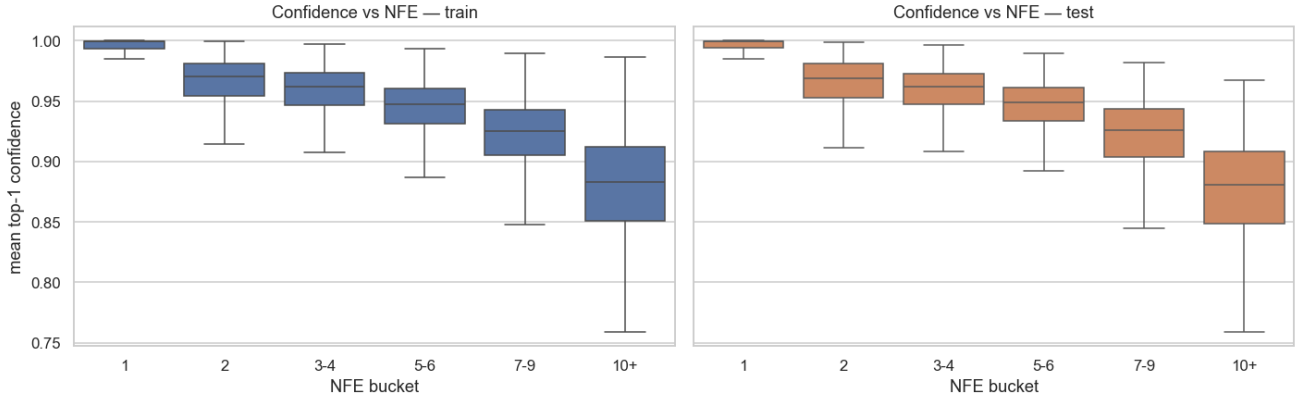


Figure 3: Mean top-1 confidence per block, bucketed by the NFE actually spent on the block, train (left) vs. test (right). Boxes show median and IQR; whiskers extend to $1.5 \times \text{IQR}$.

Figure 3 confirms that NFE is allocated where the model is least sure. Single-NFE blocks sit tightly at confidence ≈ 1.0 (the easy delimiter and high-certainty content blocks the scheduler commits immediately), and median confidence falls monotonically as the NFE bucket grows: ~ 0.97 at 2 NFE, ~ 0.95 at 5–6, ~ 0.92 at 7–9, and ~ 0.88 in the 10+ bucket. The spread also widens with NFE, so high-compute blocks are not just lower confidence on average but also more variable. The train and test panels are visually indistinguishable, supporting the use of confidence statistics as inputs to the next-tuple predictor.

4 Method

4.1 Initial CPD-Based Pipeline

The first pipeline was:

Dream/LLaDA trace collection $\rightarrow$ token-level feature extraction $\rightarrow$ CPD segmentation
 $\rightarrow$ segment tuple construction $\rightarrow$ next-tuple prediction.

The token-level features included stabilizing probability, stabilizing margin, stabilizing entropy, and stabilizing refinement step. CPD was run with PELT or kernel CPD using configurable cost, penalty, minimum segment length, and smoothing window.

Change-point detection (CPD) is the task of finding positions in a sequence where the underlying behavior changes. In our setting, the sequence is not the text itself but a token-level feature curve, such as stabilizing entropy or stabilizing probability across the generated output. A detected change point marks a candidate boundary between two decoding phases: for example, a high-confidence span followed by a lower-confidence reasoning span.

PELT (Pruned Exact Linear Time) is a dynamic-programming CPD algorithm that searches for segment boundaries by minimizing a total objective: the within-segment fit cost plus a penalty for adding more segments. The penalty controls how aggressively the algorithm splits the trace. A low penalty produces many short phases, while a high penalty keeps only the strongest regime changes. In our experiments, PELT with L2 or normal costs was useful for quick scalar-feature analysis.

Kernel CPD is a more flexible CPD variant that compares segments in an implicit feature space defined by a kernel, such as RBF, linear, or cosine. Instead of assuming that a simple mean or variance shift explains the boundary, kernel CPD can detect broader distributional changes in the feature trajectory. This makes it useful when phase shifts are nonlinear or subtle, though it can also be more sensitive to kernel choice and hyperparameters.

This pipeline was useful for analysis because it showed that token trajectories often contain visible regime changes. However, it also introduced a label-construction dependency: CPD boundaries changed with feature choice and penalty, while the downstream scheduler ultimately needed block-level controls.

4.2 Revised Block-Native Pipeline

The revised direction is:

AdaBlock/block-native traces $\rightarrow$ block tuple sequence construction
 $\rightarrow$ next-tuple prediction $\rightarrow$ tuple-conditioned unmasking.

This design keeps the training labels aligned with the scheduler. Instead of inferring block boundaries from token-level curves, the trace itself provides block size and stabilization/refinement behavior. CPD remains valuable as an analysis tool, but it is no longer the preferred production training path.

This revision moved the project toward a predictor trained on explicit block-level supervision rather than CPD-derived pseudo-boundaries.

4.3 Phase Predictor

4.3.1 Regression-Based Predictor

The first version of the phase predictor treated next-tuple prediction as a regression problem. Given a context window of previous tuples, the model predicted continuous values for the next control tuple, typically `(block_size, refinement_steps)`. Inputs were normalized using training-set statistics, the model produced one real-valued output per tuple field, and inference rounded the outputs back to non-negative integer scheduler controls.

This regression framing was simple and flexible: adding a new output field only required changing the tuple size, and mean-squared error gave a direct training objective. However, due to the mean-squared error, the model tended to produce outputs that were not well-aligned with the discrete nature of the controls. This motivated the current implementation, which keeps the Transformer encoder but changes the output heads and losses.

ADD A PICTURE IF POSSIBLE

4.3.2 Current Classification/Ordinal Transformer

The currently implemented `phase_predict.model.PhaseTransformer` is a compact Transformer encoder with two task-specific heads. Each input item is a window of previous phase tuples or extended feature tuples with shape

`(batch, window_size, input_tuple_size)`.

The model first projects each input tuple into a hidden dimension, adds fixed sinusoidal positional encodings, and passes the sequence through a stack of Transformer encoder layers. The implementation uses batch-first tensors, pre-normalized encoder layers, a feed-forward width of $4d$, and the final context position as the pooled representation.

From this final representation, the model predicts two outputs:

- a block-size classification head that maps the final hidden state to logits over block sizes $1, \dots, K_b$;
- a stabilization ordinal head that maps the final hidden state concatenated with block logits to K_s threshold logits for the maximum stabilization step.

The stabilization head is conditioned on the block prediction by concatenating the final hidden state with the block logits before predicting ordinal thresholds. Training uses cross-entropy loss for the block-size class and binary cross-entropy-with-logits for the ordinal stabilization vector, then sums the two losses. The dataset encodes the block target as `block_size - 1` and encodes the stabilization target as a prefix vector: if the target stabilization step is s , the first s thresholds are set to one.

At inference time, the predictor applies `argmax` to the block logits and adds one to recover the block size. It converts stabilization logits with a sigmoid and counts thresholds above 0.5 to recover the predicted stabilization/refinement count. The current schema defaults to 128 block classes and 83 stabilization thresholds, based on observed trace ranges. It still supports multi-feature inputs through extended tuples and configurable fields such as block size, NFE, maximum stabilization step, confidence statistics, digit fraction, and delimiter fraction.

ADD A PICTURE IF POSSIBLE

4.4 Tuple-Conditioned Diffusion Unmasking

The tuple-conditioned unmasking loop is implemented using LLaDA model. We keep a history of realized block outcomes as extended tuples, use a seed tuple for the first block, and then queries the trained phase predictor using the recent padded history. The predicted tuple is converted into a concrete scheduled block: the block-size component determines the active span to decode, and the refinement/stabilization component determines the maximum number of LLaDA forward passes allowed for that span. The scheduler clips these values by the

remaining generation length, maximum block length, global step budget, and a minimum refinement floor. In this way, the phase predictor becomes a direct controller for diffusion unmasking rather than just an offline trace model.

Inside each scheduled block, LLaDA repeatedly predicts token IDs for masked positions and transfers tokens according to a confidence-threshold policy. Under low-confidence remasking, only masked positions whose predicted token confidence exceeds the threshold are unmasked, while at least one token is forced to transfer so generation cannot stall. If the block reaches its budgeted refinement step before all positions are confidently unmasked, the implementation force-commits the remaining masked positions to keep decoding bounded. After the block finishes, the scheduler records the realized block size, actual NFE, mean and minimum confidence, digit fraction, and delimiter fraction, so the next prediction is conditioned on what actually happened during decoding. Prefix-cache and dual-cache variants preserve the same tuple-conditioned scheduling semantics while reducing repeated computation.

5 Experiments

5.1 Trace and CPD Analysis

The `phase_cpd` experiments used curated Dream/LLaDA-style traces from the local real-trace directory. The CPD UI and scripts support stabilizing entropy, margin, probability, and refinement-step features.

Finding. Stabilizing probability and entropy often reveal interpretable blocks: high-confidence spans tend to correspond to easy copied or syntactically predictable regions, while lower-confidence/noisier spans tend to align with reasoning or code-construction regions. Stabilizing refinement step was less informative in several traces because it often behaved monotonically, although spikes sometimes marked harder reasoning transitions.

Design consequence. CPD is useful for diagnosing phase-like behavior, but its sensitivity to smoothing and penalty settings made it less attractive as the sole source of training labels.

ADD PICTURES

5.2 Ablation Results

6 Preliminary Findings

1. dLLM traces do show useful stabilization structure. CPD plots over stabilizing probability and entropy reveal plausible phase boundaries in algebra and coding examples.
2. CPD is better as an analysis instrument than as a mandatory training-label generator. Hyperparameters can move boundaries, and block-native traces avoid that ambiguity.
3. The learned predictor path is viable. The codebase supports Transformer training, checkpointing, inference, and hyperparameter ablation over tuple sequences.
4. Richer tuple features are likely important. The project moved from two-field tuples toward multi-feature inputs that include NFE, stabilization statistics, confidence statistics, digit fraction, and delimiter fraction.

7 Evaluation

8 Limitations

9 Conclusion

PAG frames adaptive dLLM decoding as a learned next-control prediction problem. The project began with token-level trace analysis and CPD segmentation, then shifted toward block-native AdaBlock-style traces after observing that CPD adds avoidable ambiguity to label construction. The codebase now contains a working

staged architecture, trace analysis tools, tuple extraction, Transformer predictor training, multi-feature support, and saved ablation outputs. The remaining work for the final report is to consolidate experiments under one evaluation protocol and document the end-to-end scheduler results.

References

- J. Ye, Z. Xie, L. Zheng, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong. Dream 7B: Diffusion large language models. *arXiv:2508.15487*, 2025.
- S. Nie, F. Zhu, Z. You, X. Zhang, J. Ou, J. Hu, J. Zhou, Y. Lin, J.-R. Wen, and C. Li. Large language diffusion models. *arXiv:2502.09992*, 2025.
- G. Lu, H. M. Chen, Y. Karashima, Z. Wang, D. Fujiki, and H. Fan. AdaBlock-dLLM: Semantic-aware diffusion LLM inference via adaptive block size. *ICLR*, 2026.
- M. Arriola, A. Gokaslan, J. T. Chiu, Z. Yang, Z. Qi, J. Han, S. S. Sahoo, and V. Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. *ICLR (Oral)*, 2025.
- C. Wu, H. Zhang, S. Xue, Z. Liu, S. Diao, L. Zhu, P. Luo, S. Han, and E. Xie. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. *ICLR*, 2026.
- S. Gong, S. Agarwal, Y. Zhang, J. Ye, L. Zheng, M. Li, C. An, P. Zhao, W. Bi, J. Han, H. Peng, and L. Kong. Scaling diffusion language models via adaptation from autoregressive models. *ICLR*, 2025.